LAPORAN PRAKTIKUM STRUKTUR DATA DAN ALGORITMA

Disusun oleh:

Nama : **Sayyid Al Kautsar**

Nim : **2025573010090**

Kelas : **TI 1D**

Jurusan : **Teknologi Informasi Dan Komputer**

Prodi : **Teknik Informatika**

Dosen Pengajar : **M.Reza Zulman,SST.M.Sc**

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Segala puji bagi Tuhan Yang Maha Esa atas selesainya laporan praktikum Struktur Data Dan Algorima ini. Laporan ini merupakan rangkuman dari hasil pengamatan dan analisis yang dilakukan selama kegiatan praktikum .

Terima kasih penulis ucapkan kepada seluruh pihak yang telah membantu, terutama dosen dan rekan rekan praktikum atas arahan dan ilmunya.

Penulis memohon maaf apabila terdapat kekurangan dalam penyusunan laporan ini. Semoga hasil praktikum ini dapat memberikan wawasan baru bagi kita semua.

Lhokseumawe, 28/04/2026

DAFTAR ISI

KATA PENGANTAR .....

DAFTAR ISI .....

BAB I: PENDAHULUAN

1.1 Latar Belakang .....

1.2 Rumusan Masalah .....

1.3 Tujuan Praktikum.....

BAB II: DASAR TEORI

2.1 Pengertian Intro Kompleksitas.....

2.2 Destructuring Spread .....

2.3 Space Complexity .....

2.4 Refactor .....

BAB III: METODE PRAKTIKUM

3.1 Alat dan Bahan .....

3.2 Langkah-Langkah Praktikum .....

BAB IV: PENUTUP

4.1 Kesimpulan.....

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi algoritma merupakan aspek penting yang harus diperhatikan. Seiring dengan bertambahnya jumlah data, algoritma yang kurang efisien dapat menyebabkan waktu eksekusi yang lama serta penggunaan memori yang besar. Oleh karena itu, diperlukan suatu metode untuk menganalisis dan mengukur kinerja algoritma secara sistematis. Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity), serta mampu membandingkan efisiensi beberapa algoritma yang berbeda. Dengan demikian, mahasiswa dapat menulis kode yang lebih efektif dan efisien dalam menyelesaikan suatu permasalahan.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Mengapa analisis efisiensi algoritma penting dalam pemrograman?
3. Bagaimana cara menentukan kompleksitas waktu menggunakan Big O Notation?
4. Bagaimana cara menganalisis kompleksitas ruang (space complexity)?
5. Bagaimana membandingkan performa beberapa algoritma berdasarkan efisiensinya?

1.3 Tujuan Praktikum

1. Memahami pentingnya analisis efisiensi algoritma.
2. Menjelaskan konsep time complexity dan space complexity.
3. Mampu membaca dan menuliskan notasi Big O seperti O(1), O(n), O(log n), dan $O(n^2)$.
4. Menganalisis kompleksitas dari berbagai struktur kode seperti loop dan nested loop.
5. Membandingkan performa algoritma secara empiris menggunakan pengukuran waktu eksekusi.

BAB II DASAR TEORI

2.1 Pengertian Intro Kompleksitas

Berisi pengenalan tentang kompleksitas algoritma, yaitu cara mengukur seberapa efisien suatu algoritma berdasarkan waktu eksekusi. Biasanya membandingkan beberapa algoritma seperti O(1), O(n), dan O(n^2).

Contoh kodingannya:

```javascript
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log('${label}: ${selesai - mulai} ms');
}
const N = 100_000;

function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++) total += i;
    return total;
}

function jumlahkanRumus(n) {
    return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
    return pasangan;
}

const data = Array.from({length: 5000}, (_, i) => i);
console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('0(1)  jumlahkaRumus ', () => jumlahkanRumus(N));
ukurWaktu('0(n)  jumlakanLinear ', () => jumlahkanLinear(N));
ukurWaktu('0(n) cariPasangan   ', () => cariPasangan(data));

console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

2.2 Destructuring Spread

Membahas konsep destructuring dan spread operator dalam JavaScript, yaitu teknik untuk mengambil nilai dari array/object serta menggabungkan atau menyalin data dengan cara yang lebih ringkas.

Contoh kodingannya:

```javascript
function ambilPertama(arr)  { return arr[0]; }
function tambahkanItem(arr, item) { arr.push(item); }
function isGenap(n) { return n % 2 === 0; }

function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(`  Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
    return a;
}
```

```javascript

function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2);
}

console.log('=== O(1) — selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50]));
console.log(isGenap(42));

console.log('\n=== O(log n) — Binary Search ===');
const sorted = Array.from({length:1_000_000},(_,i)=>i);
binarySearch(sorted, 731_452);

console.log('\n=== O(n) — Linear Search ===');
console.log('Max dari 1000 elemen:',
cariMax(Array.from({length:1000},()=>Math.random()*1000|0)));

console.log('\n=== O(n²) — Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log('\n=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');
console.log('\nWaktu fib(35) O(2ⁿ):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');

```

```javascript
console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
    }

t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

2.3 Space Complexity

Menjelaskan tentang space complexity, yaitu jumlah memori yang digunakan oleh algoritma selama proses berjalan, apakah kecil (O(1)) atau bertambah sesuai data (O(n)).

Contoh kodingannya:

```
function jumlahArray(arr) {
    let total = 0;
    for (const x of arr) total += x;
    return total;
}

function duplikasiArray(arr) {
    const baru = [];
    for (const x of arr) baru.push(x * 2);
    return baru;
}

function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1);
}

function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i;
    return hasil;
}
```

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array :', jumlahArray(arr));
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));

function hitungUnik(arr) {
    const seen = new Set(); //

for (const x of arr) seen.add(x);
return seen.size;
}
const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik:', hitungUnik(dataAcak));
```

2.4 Refactor

Berisi proses refactoring, yaitu memperbaiki kode agar lebih efisien tanpa mengubah hasilnya, biasanya dengan mengurangi kompleksitas dari yang lambat (misalnya $O(n^2)$) menjadi lebih cepat ($O(n)$).

Contoh kodingannya:

```javascript
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
        for (let j = i+1; j < arr.length; j++)
        if (arr[i] === arr[j]) return true;
    return false;
}

function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
        seen.add(x);
    }
    return false;
}

function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}

```

```javascript
function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}

const besar = Array.from({length: 20000}, () => Math.floor(Math.random() *1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat Lambat O(n2) :', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat Cepat O(n) :', Date.now()-t, 'ms');

const kata = ['a', 'b', 'a', 'c', 'b', 'a', 'd'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
```

BAB III: METODE PRAKTIKUM

3.1 Alat dan Bahan

Alat:

1. Komputer/Laptop
2. Keyboard & Mouse
3. Akses Internet

Bahan:

1. Git Bash
2. Visual Studio Code

3.2 Langkah-Langkah Praktikum

1. Membuka Git Bash sebagai terminal kerja.
2. Berpindah ke direktori tempat repository berada dengan perintah cd
3. Masuk ke folder repository yang sudah di-clone,
4. Tuliskan nama repository kamu
5. Membuat folder baru untuk praktikum dengan mkdir.
6. Menginisialisasi NodeJS dengan perintah npm init –y
7. Buka folder yang udah di tambah di git di vs code
    a. Buka vs code arahkan cursor ke file cari open folder
    b. Dan cari folder yang di buat tadi di gitbash
    c. Kembali ke gitbash buat file di dalam folder tersebut menggunakan touch
    d. Cek di vs code otomatis bakalan adaa file dalam folder pratikum-5

- **Tugas-1.js**

  Input:

```javascript

function intersectionN2(arr1, arr2) {
    let result = [];
    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
                result.push(arr1[i]);
            }
        }
    }
    return result;
}
function intersectionSet(arr1, arr2) {
    const set2 = new Set(arr2);
    const result = [];

    for (let num of arr1) {
        if (set2.has(num) && !result.includes(num)) {
            result.push(num);
        }
    }
    return result;
}
function groupAnagrams(arr) {
    const map = {};

    for (let word of arr) {
        let key = word.split('').sort().join('');
        if (!map[key]) {
            map[key] = [];
        }
        map[key].push(word);
    }

    return Object.values(map);
}
function checkN3(arr) {
    const n = arr.length;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            for (let k = 0; k < n; k++) {
                if (arr[i] + arr[j] === arr[k] * arr[k]) {
                    return true;
                }
            }
        }
    }
    return false;
}
function checkNlogN(arr) {
    const set = new Set(arr);

    for (let c of arr) {
        let target = c * c;

        for (let a of arr) {
            let b = target - a;
            if (set.has(b)) {
                return true;
            }
        }
    }
    return false;
}
```

```javascript
function measure(label, fn) {
    console.time(label);
    fn();
    console.timeEnd(label);
}
const arr1 = [1, 2, 3, 4, 5];
const arr2 = [3, 4, 5, 6, 7];

const words = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'];

const nums = [1, 2, 3, 4, 5];

// Jalankan
console.log("Intersection O(n^2):", intersectionN2(arr1, arr2));
console.log("Intersection O(n):", intersectionSet(arr1, arr2));

console.log("Anagram:", groupAnagrams(words));

console.log("Check O(n^3):", checkN3(nums));
console.log("Check O(n log n):", checkNlogN(nums));

// Ukur waktu
measure("Intersection N2", () => intersectionN2(arr1, arr2));
measure("Intersection Set", () => intersectionSet(arr1, arr2));
measure("Check N3", () => checkN3(nums));
measure("Check NlogN", () => checkNlogN(nums));
```

Output:

```
C:\Program Files\nodejs\node.exe .\tugas-1.js
> Intersection O(n^2): (3) [3, 4, 5]
> Intersection O(n): (3) [3, 4, 5]
> Anagram: (3) [Array(3), Array(2), Array(1)]
Check O(n^3): true
Check O(n log n): true
Intersection N2: 0.05712890625 ms
Intersection N2: 0.099ms
Intersection Set: 0.036865234375 ms
Intersection Set: 0.101ms
Check N3: 0.052001953125 ms
Check N3: 0.124ms
Check NlogN: 0.044189453125 ms
Check NlogN: 0.105ms
```

- **Tugas-2.js**

  Input:

```javascript
function fn_O1(n) {
    return n + 1;
}

function fn_On(n) {
    let sum = 0;
    for (let i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}
function fn_OnLogN(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            count++;
        }
    }
    return count;
}
function fn_On2(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            count++;
        }
    }
    return count;
}
```

```javascript
function measure(fn, n) {
    const start = performance.now();
    fn(n);
    const end = performance.now();
    return (end - start).toFixed(4);
}
function benchmarkSemua(ukuranData) {
    console.log("Ukuran\tO(1)\tO(n)\tO(n log n)\tO(n^2)");

    for (let n of ukuranData) {
        let t1 = measure(fn_O1, n);
        let t2 = measure(fn_On, n);
        let t3 = measure(fn_OnLogN, n);
        let t4 = measure(fn_On2, n);

        console.log(`${n}\t${t1}ms\t${t2}ms\t${t3}ms\t\t${t4}ms`);
    }
}
benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Output:

```
C:\Program Files\nodejs\node.exe .\tugas-2.js
Ukuran  O(1)     O(n)     O(n log n)      O(n^2)
100     0.0235ms         0.0369ms        0.0455ms                0.4025ms
500     0.0047ms         0.0325ms        0.1157ms                1.2601ms
1000    0.0042ms         0.1332ms        0.0954ms                2.0518ms
5000    0.0013ms         0.0052ms        0.1362ms                28.0128ms
10000   0.0025ms         0.0178ms        0.6110ms                99.2387ms
```

BAB IV: PENUTUP

4.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa pemahaman mengenai kompleksitas algoritma merupakan aspek fundamental dalam pengembangan perangkat lunak yang efisien. Melalui kegiatan ini, telah dipelajari perbedaan mendasar antara *time complexity* dan *space complexity*, di mana keduanya menjadi tolok ukur utama dalam menentukan seberapa optimal suatu kode program saat menangani data dalam jumlah besar. Penggunaan notasi Big O memungkinkan identifikasi tingkatan efisiensi kode, mulai dari yang bersifat konstan hingga eksponensial, sehingga pengembangan logika program dapat dilakukan dengan lebih terukur.

Selain itu, praktikum ini juga menunjukkan bahwa teknik *refactoring* atau perbaikan struktur kode memegang peranan penting dalam meningkatkan performa aplikasi tanpa mengubah fungsionalitas aslinya. Dengan memanfaatkan fitur modern seperti *destructuring* dan *spread operator* pada JavaScript, penulisan kode menjadi lebih ringkas dan efektif. Secara keseluruhan, integrasi antara pemahaman teoritis mengenai kompleksitas dan kemampuan teknis dalam mengoptimalkan kode sangat diperlukan untuk menciptakan solusi pemrograman yang tidak hanya benar secara logika, tetapi juga hemat sumber daya waktu dan memori.